

✓ Optical Flow

Optical Flow is technique used to extract motion from multiple images, in practice we usually use only two frames of a video.

With Optical Flow, we compute a motion vector (u, v) which encodes de displacement in x and y directions between two consecutive frames.

```
!pip install git+https://github.com/Guillem96/spynet-pytorch
```

```
Collecting git+https://github.com/Guillem96/spynet-pytorch
  Cloning https://github.com/Guillem96/spynet-pytorch to /tmp/pip-req-build-56i5vvup
  Running command git clone -q https://github.com/Guillem96/spynet-pytorch /tmp/pip-req-build-56i5vvup
Requirement already satisfied: torch>=1.5.0 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (1.5.1+cu101)
Requirement already satisfied: torchvision>=0.6.0 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (0.6.1+cu101)
Requirement already satisfied: Pillow>=7.0.0 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (7.0.0)
Requirement already satisfied: scikit-image>=0.16.2 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (0.16.2)
Requirement already satisfied: matplotlib>=3.1.2 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (3.2.2)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.6/dist-packages (from spynet==1.0) (7.1.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.6/dist-packages (from torch>=1.5.0->spynet==1.0) (1.18.5)
Requirement already satisfied: future in /usr/local/lib/python3.6/dist-packages (from torch>=1.5.0->spynet==1.0) (0.16.0)
Requirement already satisfied: PyWavelets>=0.4.0 in /usr/local/lib/python3.6/dist-packages (from scikit-image>=0.16.2->spynet==1.0) (0.16.2)
Requirement already satisfied: networkx>=2.0 in /usr/local/lib/python3.6/dist-packages (from scikit-image>=0.16.2->spynet==1.0) (2.6.3)
Requirement already satisfied: scipy>=0.19.0 in /usr/local/lib/python3.6/dist-packages (from scikit-image>=0.16.2->spynet==1.0) (1.4.1)
Requirement already satisfied: imageio>=2.3.0 in /usr/local/lib/python3.6/dist-packages (from scikit-image>=0.16.2->spynet==1.0) (2.9.0)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 in /usr/local/lib/python3.6/dist-packages (from matplotlib>=3.1.2->spynet==1.0) (2.4.7)
Requirement already satisfied: python-dateutil>=2.1 in /usr/local/lib/python3.6/dist-packages (from matplotlib>=3.1.2->spynet==1.0) (2.8.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.6/dist-packages (from matplotlib>=3.1.2->spynet==1.0) (0.11.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.6/dist-packages (from matplotlib>=3.1.2->spynet==1.0) (1.3.1)
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.6/dist-packages (from networkx>=2.0->scikit-image>=0.16.2->spynet==1.0) (4.4.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.6/dist-packages (from python-dateutil>=2.1->matplotlib>=3.1.2->spynet==1.0) (1.14.0)
Building wheels for collected packages: spynet
  Building wheel for spynet (setup.py) ... done
  Created wheel for spynet: filename=spynet-1.0-cp36-none-any.whl size=11407 sha256=ce843cf26816200d64c3f7fd924744872d5dc14f5d9c
  Stored in directory: /tmp/pip-ephem-wheel-cache-srg2ujqf/wheels/14/d3/cb/3cfe52294fea3597edd0f22f5c7a400fb5c18283229247c8f
Successfully built spynet
Installing collected packages: spynet
Successfully installed spynet-1.0
```

```
!rm -rf taxi
!curl -O ftp://ftp.ira.uka.de/pub/vid-text/image_sequences/taxi/taxi.zip
!unzip -q taxi.zip
!rm taxi.zip
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
100 1454k	100 1454k	0 0	159k 0	0:00:09	0:00:09	--:--:--	341k

```
from pathlib import Path

import torch
import cv2 as cv
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from matplotlib import animation
from IPython.display import HTML, display

import torchvision.transforms as T

import spynet
```

Formulation

Optical flow is based on the Brightness consistency assumption. The intensity in location (x, y) at time t is almost equal at time $t + 1$.

$$f(x, y, t) = f(x + dx, y + dy, t + 1)$$

$$(dx, dy) = (u, v)$$

With Taylor series we can express the equation above as follows

$$f_x u + f_y v + f_t = 0$$

Above equation is called Optical Flow equation. In it, we can find f_x and f_y , they are image gradients. Similarly f_t is the gradient along time. But (u, v) is unknown. We cannot solve this one equation with two unknown variables. So several methods are provided to solve this problem:

- Discrete optimization methods
- Differential methods such as: [Lucas-kanade](#) and [Horn-Schunck](#)
- New Deep Learning approaches like FlowNet and SpyNet

✓ Hamburg taxi sequence

Famous image sequence. Old days, around 1970, generating fast sequences of images such as videos and move them to the computer was difficult. Two researchers installed a camera on their office window, and they were continuously taking photos of the street.

```
taxis_fnames = sorted(list(Path('taxi').iterdir()))
print(f'Number of frames: {len(taxis_fnames)}')
```

Number of frames: 41

STRIDE

5

[Show code](#)

```
rand_idx = np.random.randint(len(taxis_fnames) - STRIDE, size=(1,)).item()

taxi1 = Image.open(taxis_fnames[rand_idx])
taxi2 = Image.open(taxis_fnames[rand_idx + STRIDE])
```

```
plt.subplot(121)
plt.imshow(taxi1, cmap='gray')
plt.axis('off')

plt.subplot(122)
plt.imshow(taxi2, cmap='gray')
plt.axis('off');
```



OpenCV provides an algorithm to find the optical flow. It computes the optical flow for all the points in the frame. It is based on Gunner Farneback's algorithm which is explained in [Two-Frame Motion Estimation Based on Polynomial Expansion](#).

```
flow = cv.calcOpticalFlowFarneback(np.array(taxi1),
                                   np.array(taxi2),
                                   None, 0.5, 3, 15, 3, 5, 1.2, 0)
```

We can display the displacement of each pixel using a quiver plot.

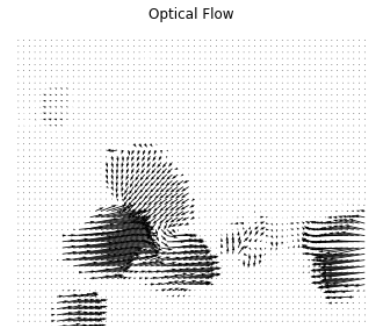
```
step = 4
plt.figure(figsize=(20, 5))

plt.subplot(131)
plt.title('Frame 1')
plt.imshow(taxi1, cmap='gray')
plt.axis('off')

plt.subplot(132)
plt.title('Frame 2')
plt.imshow(taxi2, cmap='gray')
plt.axis('off')

plt.subplot(133)
```

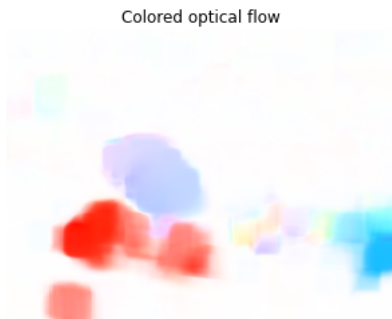
```
plt.title('Optical Flow')
plt.quiver(np.arange(0, 256, step), np.arange(190, -1, -step),
           flow[:, :step, ::step, 0], flow[:, :step, ::step, 1])
plt.axis('off');
```



Also, we can get a prettier representation of Optical Flow mapping the vector magnitudes to a color:

Where the vertical axis corresponds to the v and the horizontal axis to u

```
plt.title('Colored optical flow')
plt.imshow(spynet.flow.flow_to_image(flow))
plt.axis('off');
```



```
all_taxi_images = [np.array(Image.open(o)) for o in taxis_fnames]
optical_flows = [cv.calcOpticalFlowFarneback(im1, im2,
                                             None, 0.5, 3, 15, 3, 5, 1.2, 0)
                  for im1, im2 in zip(all_taxi_images[:-1], all_taxi_images[1:])]
optical_flows = [spynet.flow.flow_to_image(o) for o in optical_flows]
```

```
def display_video(images):
    fig = plt.figure()
    plt.axis('off')
    im = plt.imshow(images[0], cmap='gray')
    plt.close()

    def init():
        im.set_data(images[0])

    def animate(i):
        im.set_data(images[i])
        return im

    anim = animation.FuncAnimation(
        fig, animate, init_func=init, frames=len(images), interval=50)
    display(HTML(anim.to_html5_video()))

display_video(optical_flows)
display_video(all_taxi_images)
```

✓ Deep Learning Approach

To compute the optical flow with deep learning, we are going to use the *coarse to fine grained* approach defined in the SpyNet paper. Another famous NN to compute the Optical Flow is called FlowNet, but this net is much larger than the one we are using in the cells below: Spatial Pyramid Network (SPyNet) is much simpler and 96% smaller than FlowNet in terms of model parameters

```
device = torch.device('cuda' if torch.cuda.is_available() else 'gpu')

@torch.no_grad()
def predict(frames1, frames2):
    images1 = torch.stack([tfms(o.convert('RGB')) for o in frames1]).to(device)
    images2 = torch.stack([tfms(o.convert('RGB')) for o in frames2]).to(device)
    return model((images1, images2))

tfms = T.Compose([
    T.ToTensor(),
    T.Normalize(mean=[.485, .406, .456],
                 std= [.229, .225, .224])
])

model = spynet.SpyNet.from_pretrained('flying-chair')
model.to(device)
model.eval()

flow = predict([taxi1], [taxi2])[0]
plt.imshow(spynet.flow.flow_to_image(flow))
plt.axis('off');
```



```
images = [Image.open(o) for o in taxis_fnames]
all_flows = []
for im1, im2 in zip(images[1:], images[:-1]):
    flow = predict([im1], [im2])[0]
    all_flows.append(spynet.flow.flow_to_image(flow))

display_video(all_flows)
```

✓ Optical Flow for action Recognition

Most of the top performing action recognition methods use optical flow as a “black box” input. In some video action recognition classifiers that are based on 2D convolutions, which actually they are not able to extract temporal features, the optical flow is a good external feature that let us improve the classification results in front of the raw RGB images.

While it may seem intuitive to include image motion in a task related to video, often video categories in datasets can be identified from a single image. For instance, playing the guitar can easily be predicted if a guitar appears within the image. Therefore, we can think that optical flow improves the models' results is because it encodes the temporal information that 2D convolutions cannot see.

But surprisingly, in the paper “On the Integration of Optical Flow and Action Recognition” they experiment with optical flow and demonstrate that, actually, it is good because of its capability of representing scenes independently of the appearance. Also, the paper describes that as the datasets get even larger, the gap between optical flow and images accuracy decreases because the model sees the same action a lot more environments with brightness, color and objects variations.

Models that use Optical Flow as input

- [LRCN](#)

References

- [1] High Accuracy Optical Flow Method Based on a Theory for Warping: Implementation and Qualitative/Quantitative Evaluation - Mohammad Faisal and John Barron
- [2] On the Integration of Optical Flow and Action Recognition - Laura Sevilla-Lara, Yiyi Liao et.al
- [3] FlowNet: Learning Optical Flow with Convolutional Networks
- [4] Optical Flow Estimation using a Spatial Pyramid Network - Anurag Ranjan, Michael J. Black

